



المدرسة العليا لأساتذة التعليم
التقني المحمدية
جامعة الحسن الثاني بالدار البيضاء

ENSET Mohammedia

REALDEALS

E-commerce Platform and Admin Dashboard

Final Year Project Report

Field	Information
Student	Yahya Satati
Project	RealDeals Streetwear E-commerce Platform
Technologies	React, Spring Boot, MySQL
Academic Year	2025/2026
Supervisor	Mr. Abdelaziz Daaif

Institution: ENSET Mohammedia - Ecole Normale Supérieure de l'Enseignement Technique de Mohammedia
Date: 2025/2026

GitHub Repository: https://github.com/6tati/realdeals_pfe

Acknowledgements

I would like to express my deepest gratitude and sincere thanks to all my teachers at ENSET Mohammedia. Their guidance, patience, and academic support helped me build the knowledge required to complete this final year project. Every course, evaluation, and practical exercise contributed to my understanding of software development and project methodology.

A special thanks goes to my supervisor, Mr. Abdelaziz Daaif, for his support and direction during the development of the RealDeals platform. His feedback helped me organize the project, correct technical problems, improve the structure of the application, and present the work in a professional way.

I am also thankful to all the teachers who contributed to my learning in web development, databases, programming, modeling, networking, and full-stack development. Their efforts helped me understand how a real application is planned, modeled, implemented, tested, and documented.

Finally, I would like to thank my family and friends for their encouragement during the project period. Their support helped me continue working through technical problems, project pressure, debugging, and final preparation.

Abstract

This report presents the design and development of RealDeals, a full-stack e-commerce platform created for a streetwear brand. The project includes a public shopping website for customers and an administration dashboard for managing products, categories, clients, orders, sales indicators, and product images.

The application was implemented using React for the frontend, Spring Boot for the backend, and MySQL for data storage. It includes JWT authentication, role-based authorization, product image upload, cart management, checkout, order tracking, and an admin dashboard.

Keywords

E-commerce, React, Spring Boot, MySQL, JWT, Admin Dashboard, Streetwear, Product Management, Image Upload.

Table of Contents

1. General Introduction	4
2. Project Context	5
3. Objectives and Requirements	6
4. Technology Stack	7
5. System Architecture	8
6. Database and UML Design	10
7. Backend Implementation	13
8. Frontend Implementation	16
9. Admin Dashboard	18
9.3 Customer Communication, Notifications, and AI-Ready Support	20
9.4 Mobile User Interface Optimization	21
10. Testing and Validation	22
11. Challenges and Solutions	23
12. Future Improvements	24
13. Conclusion	24

1. General Introduction

E-commerce platforms have become essential for modern brands, especially in the clothing and streetwear market. A brand needs a complete digital system that allows customers to discover products, add them to a cart, place orders, and follow their purchases. At the same time, the brand owner needs an administration space to manage products, categories, clients, and orders.

RealDeals is a streetwear brand concept built around the slogan "Give to Get". The project transforms this brand idea into a web platform composed of two main parts: a public website and an admin dashboard. The public website focuses on product presentation, brand identity, and shopping. The admin dashboard focuses on management and operational control.

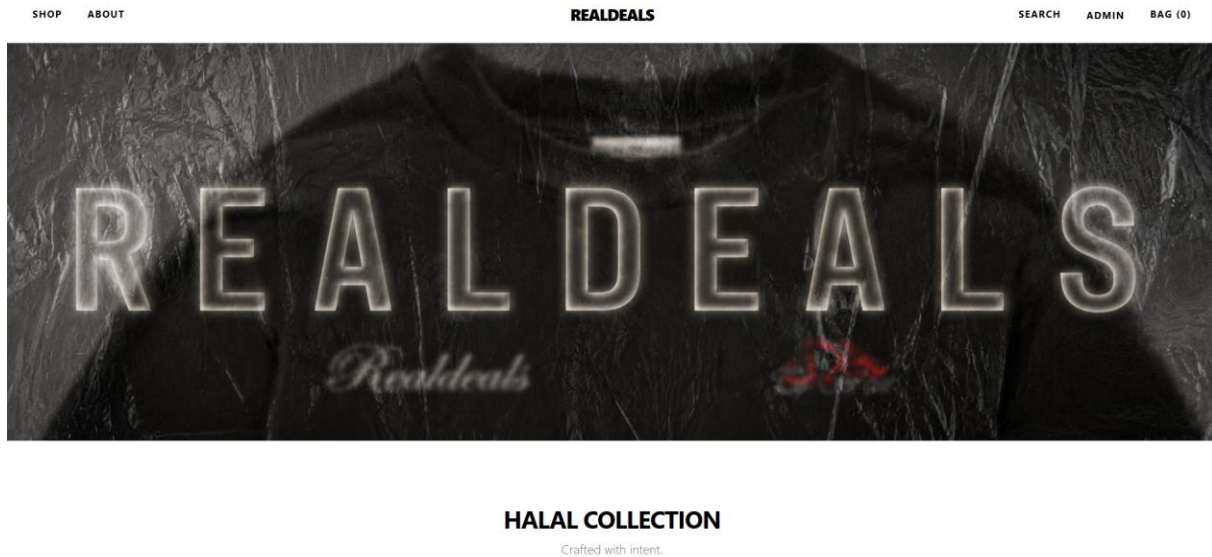


Figure 1.1: RealDeals home page and featured collection.

2. Project Context

RealDeals is positioned as a streetwear brand targeting young customers interested in simple but expressive clothing. The brand sells products such as oversized t-shirts, polo shirts, button-up shirts, caps, and accessories. Because the brand identity is visual and product-based, the website must present images clearly and create a premium shopping experience.

The project context is based on the need for a small brand management system. Many small brands start by selling through social media, but this approach becomes limited when the product catalog grows. Managing orders, stock, clients, and images manually can become difficult. The web application solves this problem by centralizing these operations.

2.1 Target Users

- Customers who browse products, add items to the cart, and place orders.
- Administrators who manage products, categories, orders, clients, and sales information.
- The brand owner who needs a professional digital presentation of the RealDeals identity.

2.2 Project Scope

The scope covers product browsing, product details, cart management, checkout, authentication, user order tracking, and administration tools. Advanced features such as online payment, delivery API integration, and advanced analytics are considered future improvements.

3. Problem Statement, Objectives, and Requirements

The main problem is how to build a reliable and easy-to-use e-commerce platform for a streetwear brand while keeping the system simple enough to be developed within the project deadline. The platform must combine a clean customer experience with a functional admin dashboard.

3.1 Main Objectives

- Create a responsive e-commerce website using React.
- Build a REST API using Spring Boot.
- Store and organize data using MySQL.
- Implement login and registration with JWT authentication.
- Provide an admin dashboard for products, categories, orders, clients, and sales.
- Allow product image uploads and multiple product images.

3.2 Functional Requirements

Requirement	Description
Product browsing	Customers can view products and open detail pages.
Cart and checkout	Customers can add items and place orders.
Authentication	Users can register, login, and receive a JWT token.
Admin management	Admins can create, edit, delete, and upload product images.
Order management	Admins can view orders and update order status.

4. Technology Stack

The RealDeals platform was built with a modern full-stack architecture. React was selected for the frontend because it allows reusable components and dynamic interfaces. Spring Boot was selected for the backend because it provides a structured way to build REST APIs and connect them to a database. MySQL was used as the relational database because the project data is structured around users, products, orders, categories, and order lines.

Layer	Technologies
Frontend	React, Vite, Tailwind CSS, React Router
Backend	Spring Boot, Spring Security, REST Controllers, JPA/Hibernate
Database	MySQL
Authentication	JWT token-based authentication
Tools	IntelliJ IDEA, VS Code, browser tests, PowerAMC

Dependency Example

```
"dependencies": {
  "react": "^19.2.7",
  "react-dom": "^19.2.7",
  "react-router-dom": "^7.18.1"
}
```

5. System Architecture

The application follows a three-layer architecture. The frontend is responsible for the user interface and communicates with the backend through HTTP requests. The backend exposes REST endpoints and handles business logic, security, and persistence. The database stores all application data.

5.1 Architecture Overview

- React frontend: pages, components, services, cart context, admin components.
- Spring Boot backend: controllers, security configuration, repositories, entities.
- MySQL database: relational model with products, users, orders, categories, and chat tables.
- JWT security: token stored in localStorage and sent in the Authorization header.

General Architecture of the RealDeals Platform

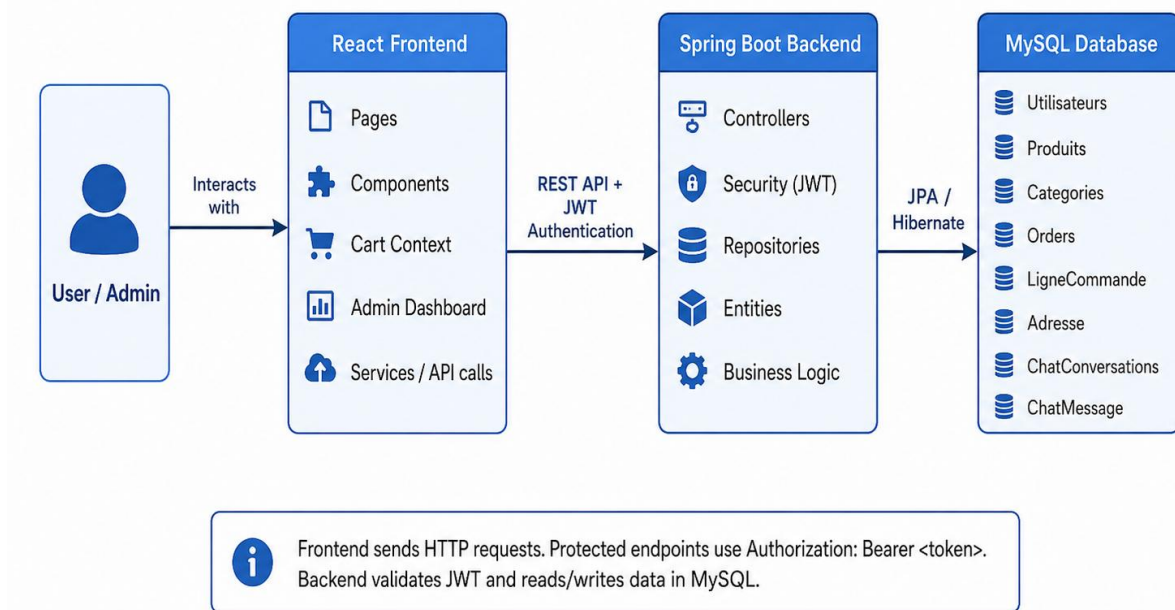


Figure 5.1: General architecture of the RealDeals platform.

5.2 Request Flow Example

1. The user logs in and receives a JWT token.
2. The frontend stores the token and user data in localStorage.
3. Protected requests include Authorization: Bearer <token>.
4. Spring Security validates the token and checks the user role.
5. The controller processes the request and returns JSON data.

Request Flow Example: Login and Protected Request

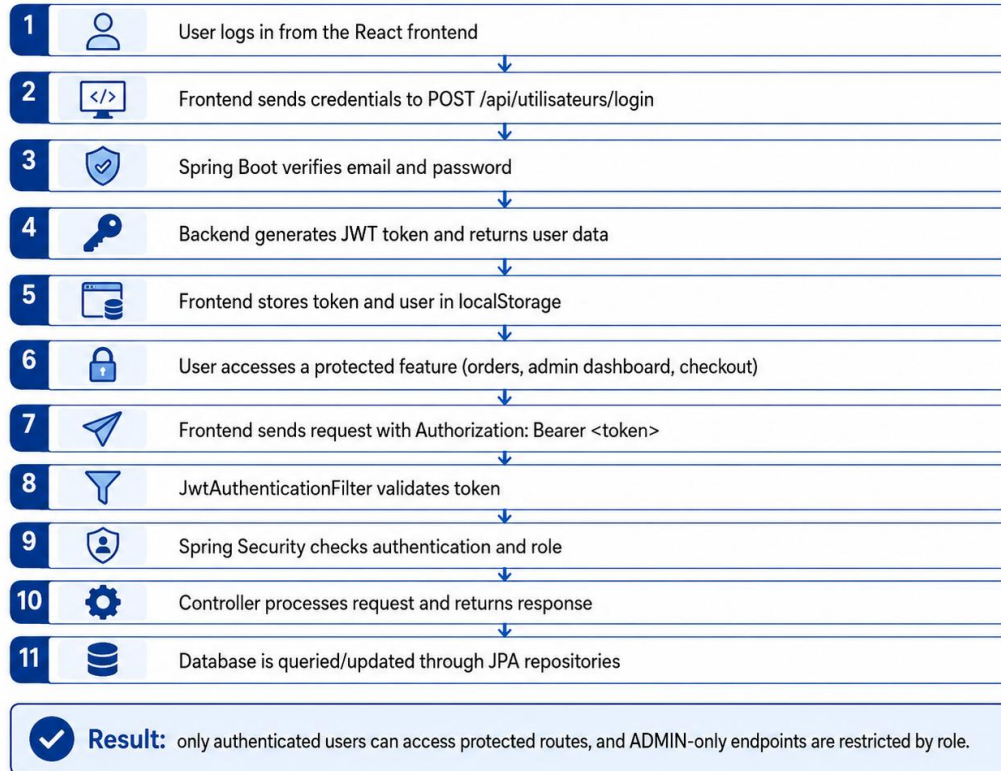


Figure 5.2: Login and protected request flow.

6. Database and UML Design

The database model was designed to represent the main business entities of the platform. The system includes users, products, categories, orders, order lines, addresses, chat conversations, and chat messages. The design supports both customer shopping operations and administration operations.

6.1 Main Entities

- Utilisateurs: stores user account information and role.
- Produits: stores product details, prices, stock, sizes, and images.
- Categories: groups products by collection or type.
- Orders: stores customer orders and their status.
- LigneCommande: stores product quantities and prices inside each order.
- Adresse: stores customer address information.
- ChatConversations and ChatMessage: prepared for client support communication.

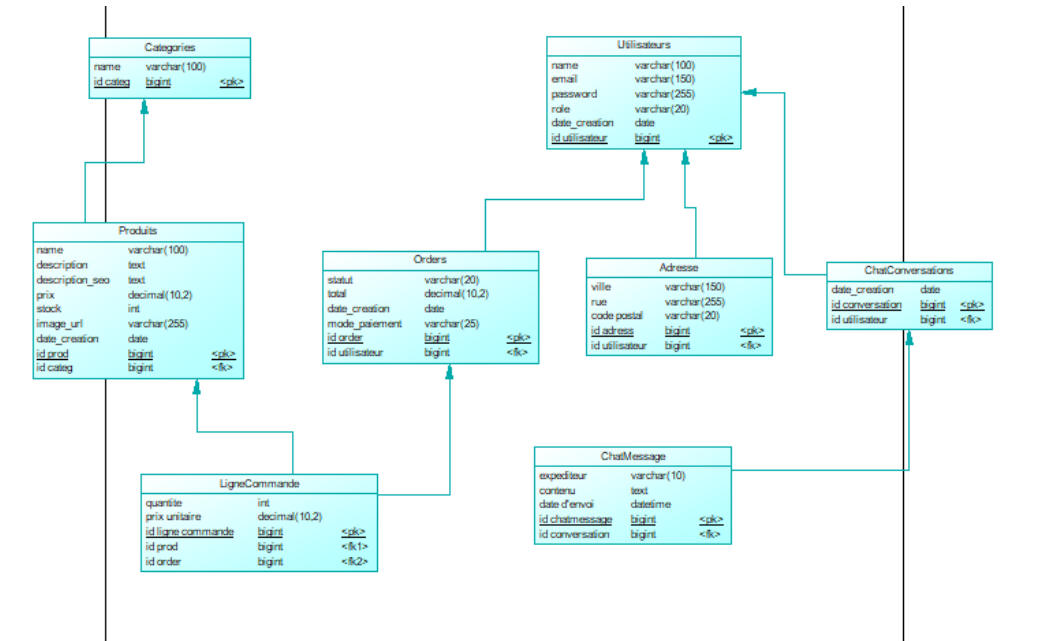


Figure 6.1: Main entities and relationships overview.

6.2 Conceptual and Logical Data Models

The conceptual model describes the business objects and relationships before implementation, while the logical model translates the design into tables, fields, primary keys, and foreign keys. These models were created in PowerAMC and used as the basis for the MySQL schema.

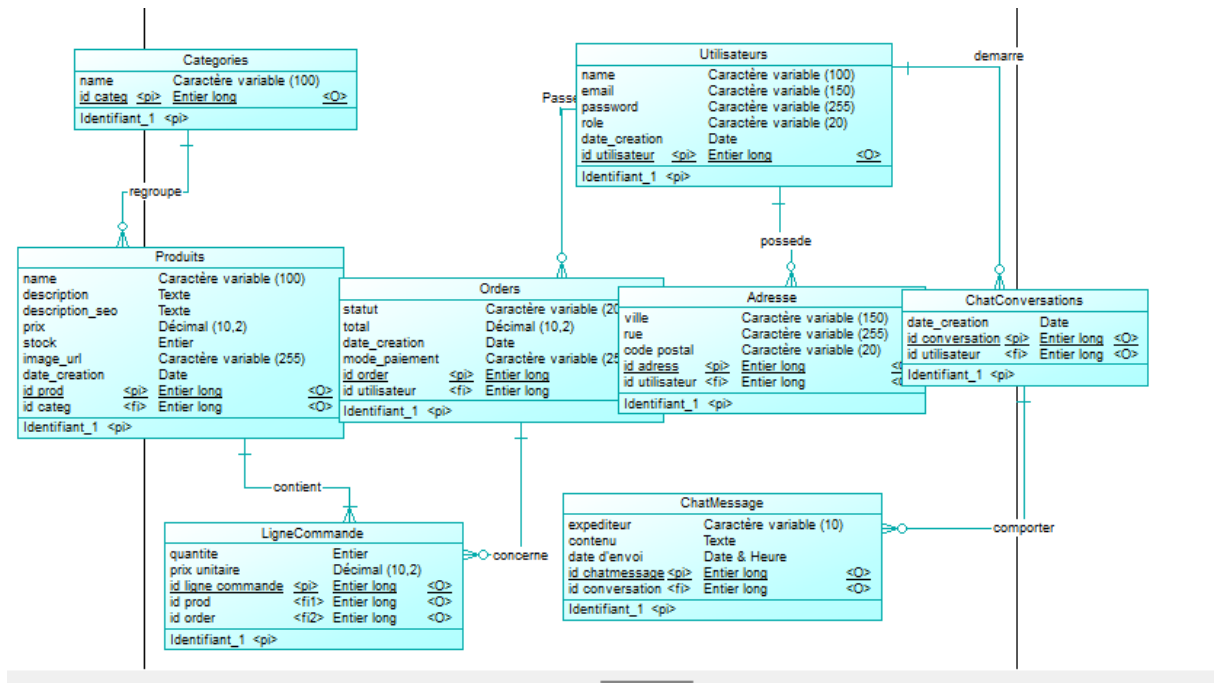


Figure 6.2: Conceptual/UML database model for RealDeals.

6.3 Database Tables and SQL Example

Table	Purpose
utilisateurs	Stores account information, password, role, and creation date.
categories	Stores product categories and collections.
produits	Stores product details, price, stock, sizes, and images.
orders	Stores order status, total amount, payment mode, phone, address, and user.
lignecommande	Stores order lines with quantity, unit price, product, and order reference.

```
CREATE TABLE produits (  
  id_prod BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  description TEXT,  
  prix DECIMAL(10,2) NOT NULL,  
  stock INT NOT NULL DEFAULT 0,  
  tailles VARCHAR(100),  
  image_url VARCHAR(255),  
  image_urls TEXT,  
  id_categ BIGINT,  
  FOREIGN KEY (id_categ) REFERENCES categories(id_categ)  
);
```

7. Backend Implementation

The backend was implemented using Spring Boot. It exposes REST endpoints under the /api path and communicates with MySQL using JPA repositories. The main backend packages include controllers, entities, repositories, and configuration classes.

7.1 Controllers and Repositories

- UtilisateursController handles registration, login, user listing, update, and delete operations.
- ProduitsController handles product retrieval, creation, update, and deletion.
- CommandesController handles order creation, order history, and status updates.
- AdminDashboardController provides simplified admin data using SQL queries for dashboard stability.
- UploadController handles product image uploads.

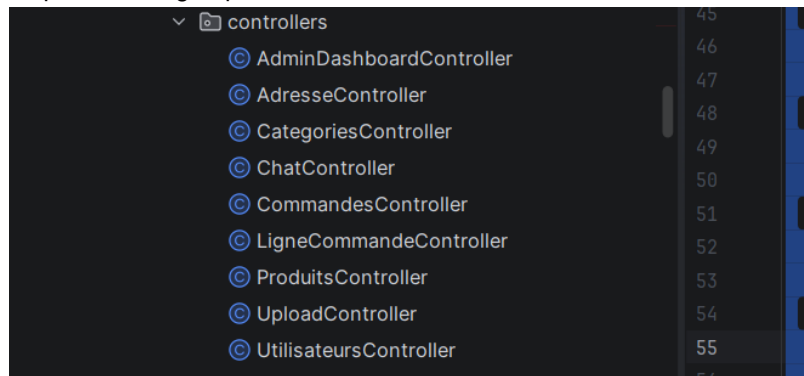


Figure 7.1: Backend controller package structure.

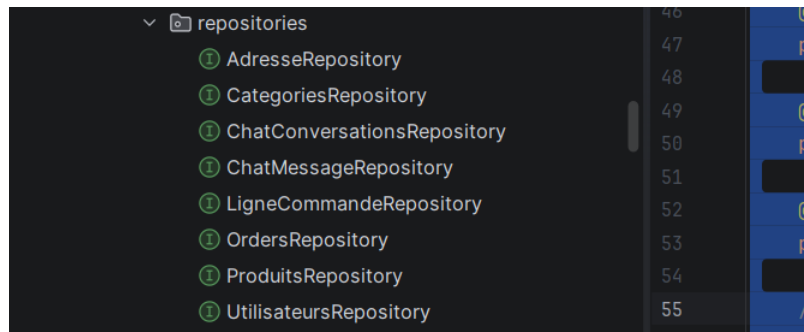


Figure 7.2: Repository package structure.

7.2 Entities and Security

Entities map Java classes to database tables using JPA annotations. Security is based on JWT tokens and role-based access. Protected routes require an Authorization header, and admin endpoints require the ADMIN role.

```
@Entity
@Table(name = "PRODUITS")
public class Produits {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idProd;
    private String name;
    private BigDecimal prix;
    private Integer stock;
    private String tailles;
    private String imageUrl;
    @Column(columnDefinition = "TEXT")
    private String imageUrls;
}

.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/commandes/**").authenticated()
.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```

7.3 Order and Image Upload Logic

The order creation process retrieves the authenticated user from the Spring Security context instead of relying on the frontend to send a user object. This prevents null user problems and avoids creating orders for another account.

For product images, the admin uploads files from the dashboard. The backend receives files using `MultipartFile`, stores them in `uploads/products`, and returns public image URLs. The first image is stored in `imageUrl` and the gallery is stored in `imageUrls`.

```
@PostMapping("/product-image")
public ResponseEntity<Map<String, String>> uploadProductImage(@RequestParam("file") MultipartFile file) {
    try {
        if (file.isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of( k1: "error", v1: "File is empty"));
        }

        Files.createDirectories(Paths.get(uploadDir));

        String originalName = file.getOriginalFilename();
        String extension = "";

        if (originalName != null && originalName.contains(".")) {
            extension = originalName.substring(originalName.lastIndexOf( str: "."));
        }

        String fileName = UUID.randomUUID() + extension;
        Path filePath = Paths.get(uploadDir, fileName);

        Files.write(filePath, file.getBytes());

        String imageUrl = "http://localhost:8080/uploads/products/" + fileName;

        return ResponseEntity.ok(Map.of( k1: "imageUrl", imageUrl));
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body(Map.of( k1: "error", v1: "Upload failed"));
    }
}
```

Figure 7.3: UploadController image upload method.

8. Frontend Implementation

The frontend was implemented with React and Vite. The project is organized into pages, components, context, and services. React Router handles navigation between pages, while the API service functions handle communication with the backend.

8.1 Main Pages

- Home page: presents the brand, banner, and featured collection.
- Products page: displays all available products.
- Product detail page: shows product images, sizes, price, stock, and add-to-cart action.
- Cart page and cart drawer: display selected products.
- Checkout page: collects address and phone information and sends the order.
- Login and Register pages: handle authentication.

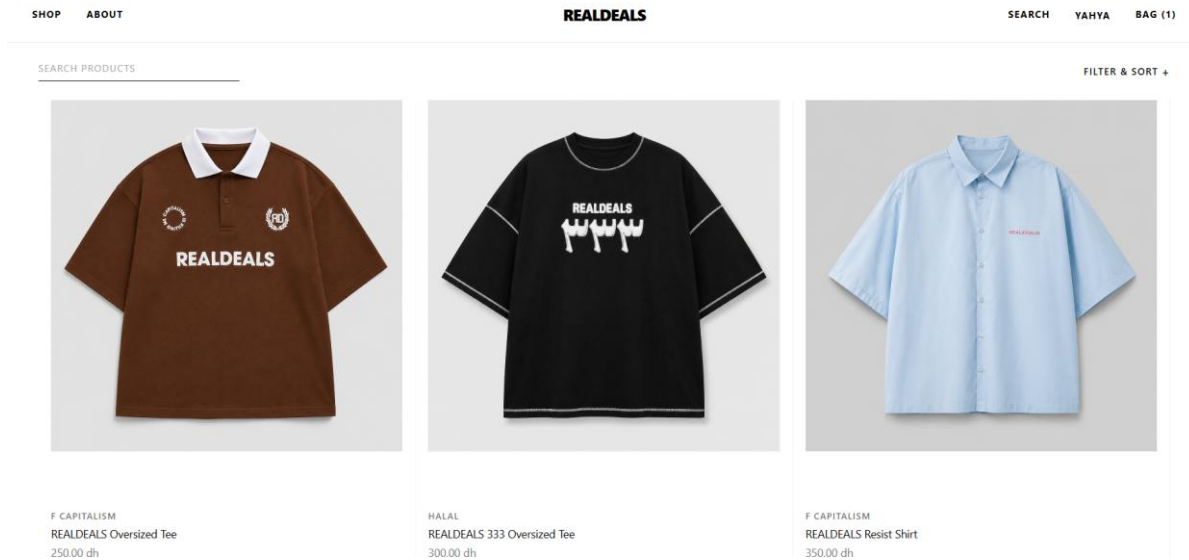


Figure 8.1: Product listing page.

8.2 User Interface and Shopping Flow

The user interface follows a minimalist streetwear e-commerce style. It uses large product images, strong typography, black and white colors, and simple spacing. The goal is to make the products stand out while keeping the shopping process clear and direct.

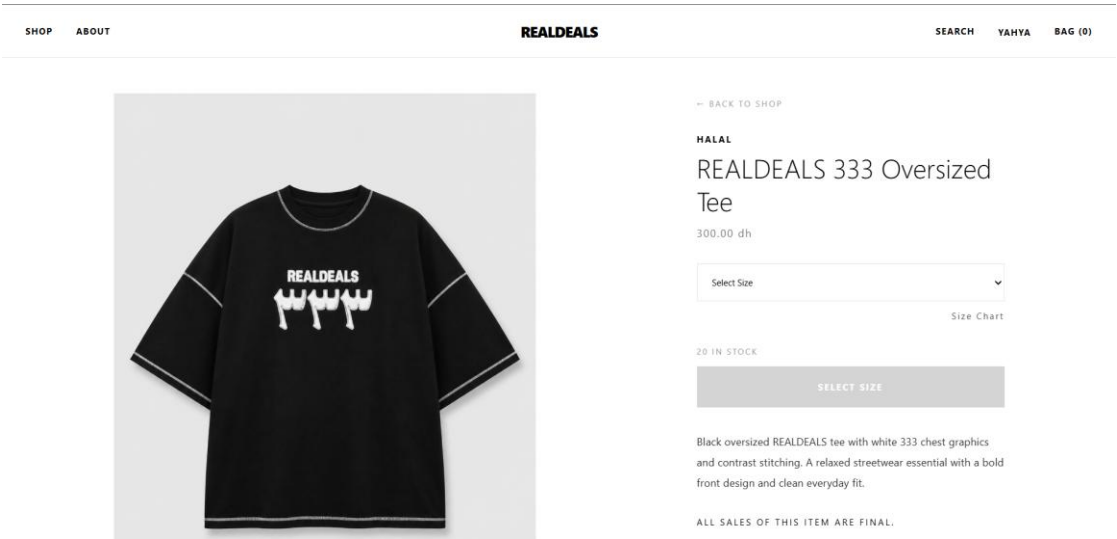


Figure 8.2: Product detail page with size selection.

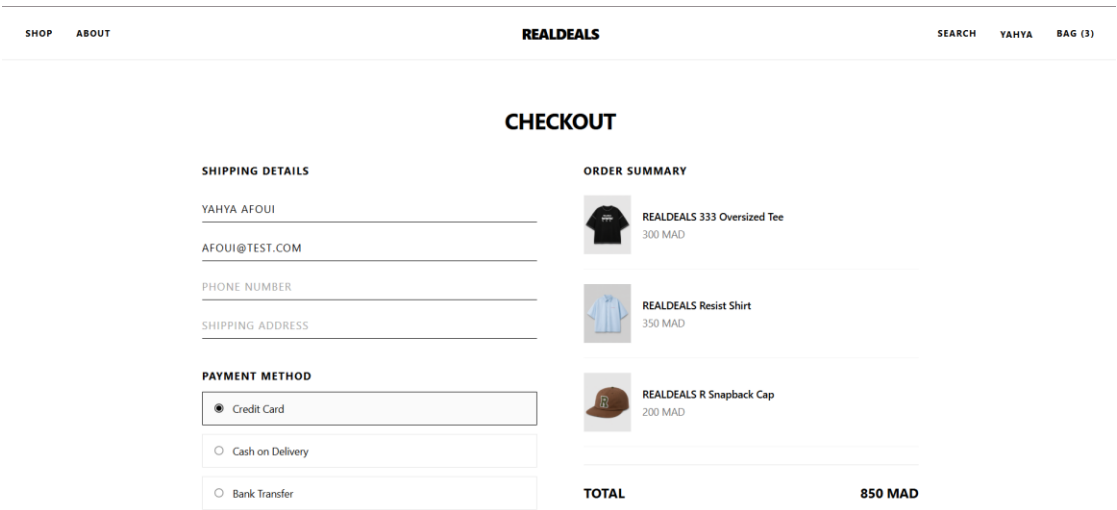


Figure 8.3: Checkout page and order summary.

9. Admin Dashboard

The admin dashboard is the management part of the system. It is protected by the ADMIN role and allows the administrator to monitor the store and manage daily operations.

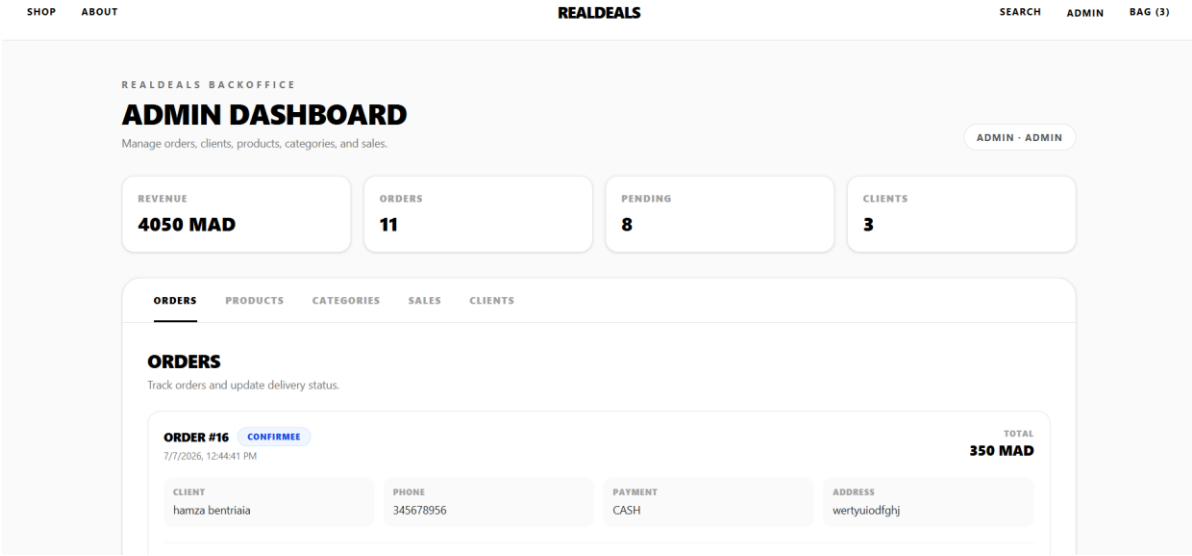


Figure 9.1: Admin dashboard overview.

9.1 Main Admin Features

- Dashboard indicators for revenue, orders, pending orders, and clients.
- Product creation, editing, deletion, and multiple image upload.
- Category management.
- Order status update.
- Client list and sales statistics.

9.2 Product, Category, Order, and Client Management

The admin can create, edit, and delete products. Product forms include name, category, price, stock, sizes, description, and multiple image upload. The admin can also create categories, view registered clients, monitor orders, and update order delivery status.

The screenshot shows the 'CATEGORIES' management interface. At the top, there's a navigation bar with 'ORDERS', 'PRODUCTS', 'CATEGORIES' (active), 'SALES', and 'CLIENTS'. Below the navigation bar, the title 'CATEGORIES' is followed by the subtitle 'Organize products by category.' A form for creating a new category is visible, with a text input field containing 'Hoodies, Caps, T-shirts...' and an 'ADD CATEGORY' button. Below this, two existing categories are shown: 'CATEGORY #2' with the name 'F CAPITALISM' and 'CATEGORY #1' with the name 'HALAL'.

Figure 9.2: Product creation and image upload form.

The screenshot shows the 'PRODUCTS' management interface. At the top, there's a navigation bar with 'ORDERS', 'PRODUCTS' (active), 'CATEGORIES', 'SALES', and 'CLIENTS'. Below the navigation bar, the title 'PRODUCTS' is followed by the subtitle 'Create, review, and delete store products.' A button 'ADD PRODUCT' is located in the top right corner. Below the subtitle, it says '6 products in catalog.' The main content is a table with the following data:

PRODUCT	CATEGORY	PRICE	SIZES	STOCK	ACTION
 REALDEALS Oversized Tee 4 images	f Capitalism	250 MAD	M, L, XL	20	EDIT DELETE
 REALDEALS 333 Oversized Tee 4 images	halal	300 MAD	M, L, XL	20	EDIT DELETE
 REALDEALS Resist Shirt 4 images	f Capitalism	350 MAD	M, L, XL	20	EDIT DELETE
 REALDEALS Halal Oversized Tee 4 images	halal	300 MAD	M, L, XL	20	EDIT DELETE
 REALDEALS Red Polo 3 images	f Capitalism	200 MAD	M, L, XL	20	EDIT DELETE

Figure 9.3: Product management table.

9.3 Customer Communication, Notifications, and AI-Ready Support

During the final improvement phase, the platform was extended with additional customer support features. These additions improve communication between customers and the brand, make order status clearer, and prepare the project for future AI-assisted shopping support.

Contact Messages Module

A contact form was connected to the backend so that customer messages are no longer only visual elements on the frontend. When a visitor sends a message from the Contact page, React sends a POST request to the Spring Boot API. The backend stores the message in the CONTACT_MESSAGES table, and the administrator can later access it from the Admin Dashboard through the Messages tab.

- Frontend page: Contact.jsx collects full name, email, subject, and message.
- Backend endpoint: POST /api/contact-messages saves the message in MySQL.
- Admin endpoint: GET /api/contact-messages allows the administrator to view support messages.
- Admin actions: mark messages as read or delete resolved messages.

Delivery Tracking and Notifications

The order module was also improved with delivery tracking and notification logic. Each order follows a status cycle: EN_ATTENTE, CONFIRMEE, EXPEDIEE, LIVREE, or ANNULEE. When the admin changes an order status from the dashboard, the backend can create a notification linked to the concerned customer. The customer can then view the delivery progress in the My Orders page and receive updates from the notification icon in the navigation bar.

Customer Contact Flow	Order Notification Flow
Contact Page -> POST /api/contact-messages -> ContactMessageController -> ContactMessageRepository -> CONTACT_MESSAGES -> Admin Messages tab	Admin changes order status -> CommandesController updates order -> Notification created -> Customer sees update in My Orders / notification menu

AI-Ready Chat Tables

The database also includes ChatConversations and ChatMessage tables. These tables were created to prepare the platform for a future AI assistant feature. The goal is to allow customers to ask questions about products, navigation, checkout, delivery, and order status. ChatConversations represents a support discussion linked to a user, while ChatMessage stores each user or assistant message. In a future version, the backend can send the customer message and product context to an AI service, save the generated answer as a BOT message, and display the conversation in the frontend.

Table	Purpose
ChatConversations	Stores one support conversation linked to a user account.
ChatMessage	Stores messages exchanged inside the conversation, with sender type USER or BOT.
Future AI Service	Generates answers using the customer question and RealDeals product/context data.

9.4 Mobile User Interface Optimization

The mobile interface was redesigned to provide a cleaner streetwear shopping experience inspired by premium fashion e-commerce layouts. The navigation bar was simplified into a menu icon, centered RealDeals logo, search icon, and bag icon. Pages were adapted to small screens with larger spacing, clear product images, readable typography, and responsive admin dashboard cards.



Figure 9.4: Mobile product listing page.

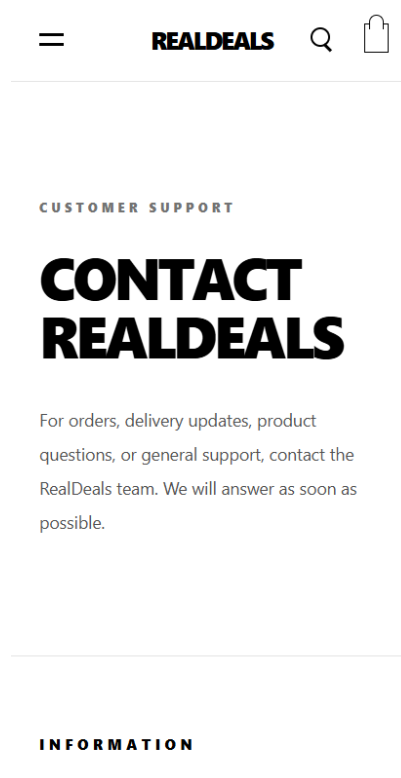


Figure 9.5: Mobile contact page.

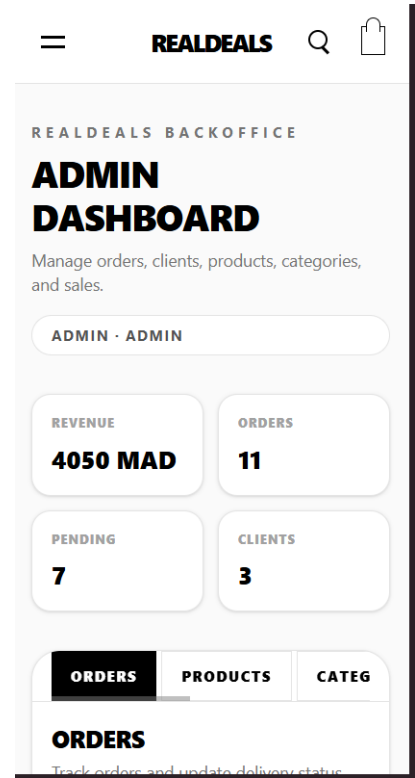


Figure 9.6: Mobile admin dashboard page.

Responsive Improvements

- Mobile navbar: menu icon, centered logo, search icon, and cart/bag icon.
- Product page: product cards are stacked vertically with large images and clear prices.
- Contact page: the support content is simplified for mobile reading and form completion.
- Admin dashboard: statistics cards and management tabs are reorganized to fit phone screens.

These changes make the platform usable from both desktop and mobile devices, which is important because many streetwear customers discover products and place orders directly from their phones.

10. Testing and Validation

Testing was performed manually through browser actions, form submissions, and API responses. The objective was to validate each main feature and verify the interaction between frontend, backend, and database.

Test Case	Action	Expected Result
Login as admin	Submit admin credentials	Token and admin data stored
Load dashboard	Open /admin	Orders, clients, and stats displayed
Create product	Submit product form with images	Product saved and image URLs stored
Checkout	Submit order with address and phone	Order created for authenticated user
Update order status	Change status in dashboard	New status saved in database
Unauthorized access	Open admin as customer	Access blocked or redirected

11. Challenges and Solutions

Challenge	Cause	Solution
ERR_INCOMPLETE_CHUNKED_ENCODING	JPA entity serialization and lazy relations	Used DTO/SQL admin endpoints and ignored recursive relations
403 Forbidden	Missing Authorization header or security rule	Added Bearer token to protected requests and configured roles
Order user null	Request body did not safely contain user	Retrieved authenticated user from SecurityContext
Image upload size	Default multipart upload limit	Configured multipart size limits in application.properties
Multiple product images	Single imageUrl field was not enough	Added imageUrls gallery field and multiple upload logic

12. Future Improvements

The current version provides the essential features needed for a functional e-commerce platform and admin dashboard. Several improvements can be added in future versions to make the platform more complete and production-ready.

- Add online payment integration with a secure payment gateway.
- Add delivery tracking and automatic order notifications.
- Add advanced analytics with charts by date, category, and product.
- Add inventory alerts for low stock products.
- Add product variants with separate stock per size and color.
- Improve client support with real-time chat.
- Deploy the application online with HTTPS.

13. Conclusion

The RealDeals project demonstrates the development of a complete full-stack e-commerce platform for a streetwear brand. The platform includes a public website for customers and an admin dashboard for internal management. It allows customers to browse products, add items to the cart, place orders, and view order history. It also allows administrators to manage products, images, categories, clients, orders, and sales indicators.

Overall, this project helped transform a brand concept into a structured application and provided practical experience in building, debugging, testing, and documenting a realistic web system.